

COMP3320/6464 High-Performance Scientific Computing

Shared Memory Parallel Programming – Pthreads

Dr Haibo Zhang
School of Computing
Australian National University



Australian
National
University

ANU College of
Engineering & Computer Science

Errors encountered on Gadi (Assignment 2)

opengl_main.cpp:76: undefined reference to `glutPostRedisplay'
opengl_main.cpp:661: undefined reference to `glutBitmapHelvetica18'
....

Solution to fix it:

1. Open CMakeLists.txt, change `${GLUT_LIBRARY}` should be `${GLUT_LIBRARIES}` in lines 64 and 68.
2. The CMakeLists.txt in GitLab has been updated. Overwrite your version with the updated version.

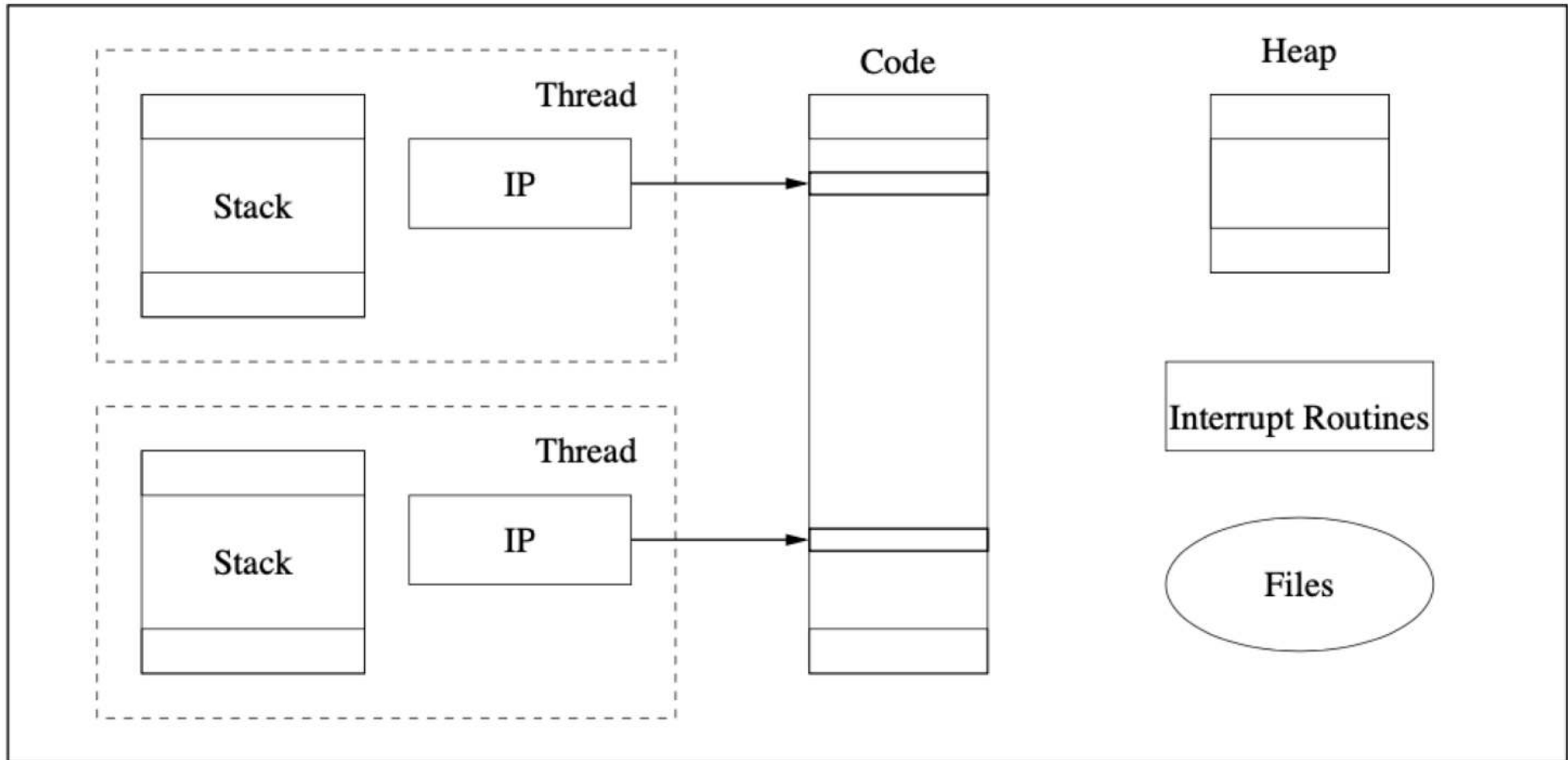
This week

- Monday: assignment 2
- Tuesday: thread memory model
thread synchronisation

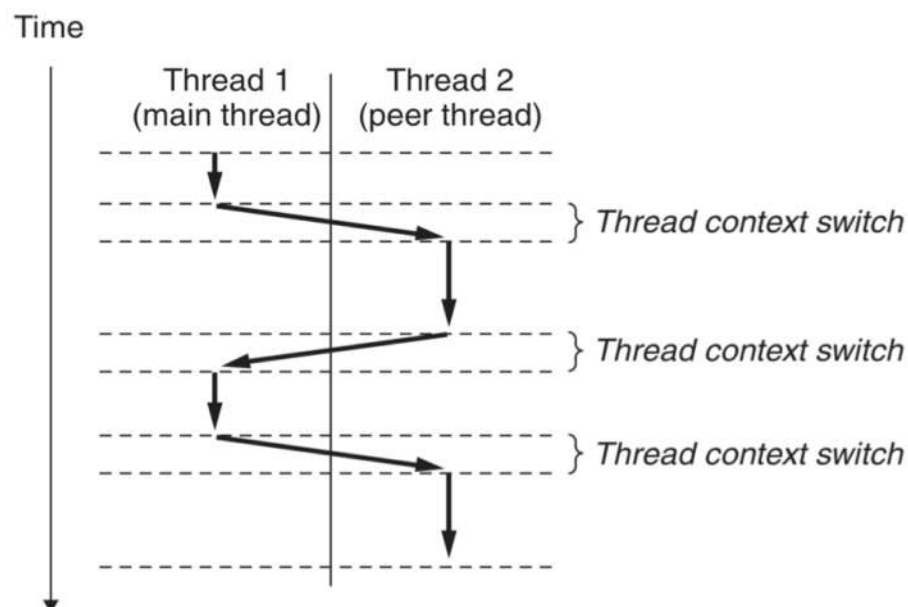
A thread is a **light-weight** sub-process within the context of a process

- Threads inherit many attributes of a the parent process (such as the process address space).
- Multiple “independent” threads can be added to an existing process rather than starting a new process.
- Each thread in the process runs concurrently (may be not at the same time), and has access to the calling process’s global data.
- Each thread has its own private attributes and call stack.
- Smallest sequence of programmed instructions to be scheduled by the OS and run as independent entities largely because threads in the same process duplicate only the bare essential resources that enable them to exist as executable code.

Roadmap



Thread Execution Model



- Each process is created with a single thread called the *main* thread.
- The main thread can create a *peer* thread, and the two run concurrently.
- If they are scheduled on the same CPU/core, the OS at some point will pass control from the main to the peer thread via a context switch.
- If they are scheduled by the OS on different cores they run in parallel.
- Threads associated with a process form a *pool* of peers, where each one can kill or wait for any other to terminate.
- Each peer can read and write the same shared data.

Creating threads

```
#include <pthread.h>
typedef void *(func)(void *);

int pthread_create(pthread_t *tid, pthread_attr_t *attr,
                  func *f, void *arg);
```

Returns: 0 if OK, nonzero on error

Terminating threads

- *Implicit* termination when its top-level thread routine returns.
- *Explicit* termination using `pthread_exit`. Explicit termination of main thread causes to wait all other peers to terminate.

```
#include <pthread.h>

void pthread_exit(void *thread_return);
```

Returns: 0 if OK, nonzero on error

- A peer thread calls `pthread_cancel` with the ID of the current thread.

```
#include <pthread.h>

int pthread_cancel(pthread_t tid);
```

Returns: 0 if OK, nonzero on error

- A peer thread calls `exit` terminating the process and all its threads.

Returning and reaping

```
#include <pthread.h>

int pthread_join(pthread_t tid, void **thread_return);
```

Returns: 0 if OK, nonzero on error

- Current thread waits for thread `tid` to terminate, blocking until it does so.
- Assigns the generic (`void *`) pointer returned by the thread routine to the location pointed to by `thread_return`.
- *Reaps* any memory resources (e.g. stack) held by the terminated thread.

Detaching threads

- A thread is either *joinable* or *detachable*.
- *Joinable*: its memory resources (such as the stack) are not freed until it is reaped by another thread.
- A detached thread cannot be reaped by other threads. Its memory resources are freed automatically by the system when it terminates.
- To avoid memory leaks, each joinable thread should either be explicitly reaped by another thread, or detached by a call to `pthread_detach`

```
#include <pthread.h>
```

```
int pthread_detach(pthread_t tid);
```

Returns: 0 if OK, nonzero on error

- Threads can detach themselves by calling `pthread_detach` with an argument of `pthread_self`.

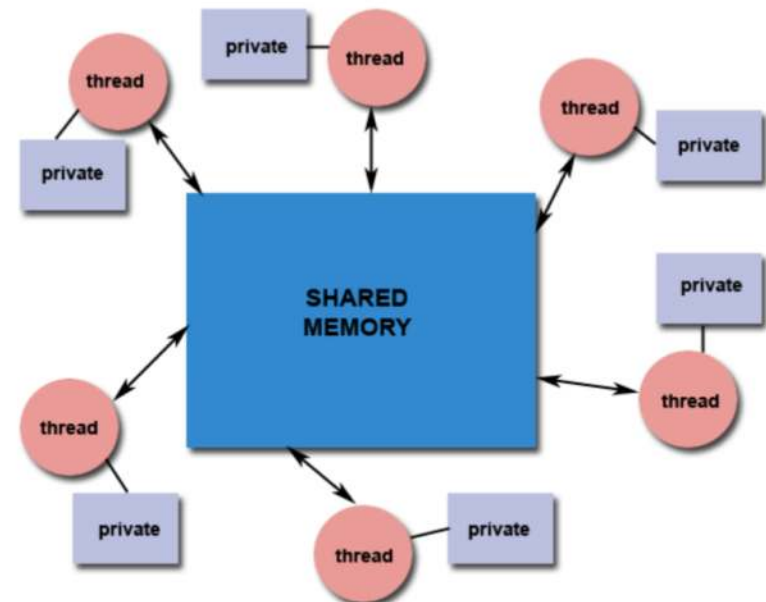
Pthreads Memory Model



The most attractive aspect of threads is the efficiency and easy with which they can share data

```
1  #include "csapp.h"
2  #define N 2
3  void *thread(void *vargp);
4
5  char **ptr; /* Global variable */
6
7  int main()
8  {
9      int i;
10     pthread_t tid;
11     char *msgs[N] = {
12         "Hello from foo",
13         "Hello from bar"
14     };
15
16     ptr = msgs;
17     for (i = 0; i < N; i++)
18         Pthread_create(&tid, NULL, thread, (void *)i);
19     Pthread_exit(NULL);
20 }
21
22 void *thread(void *vargp)
23 {
24     int myid = (int)vargp;
25     static int cnt = 0;
26     printf("[%d]: %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
27     return NULL;
28 }
```

- The price to pay is the potential for complicated bugs that do not appear in serial code.
- To write correct threaded programs a clear understanding of the sharing mechanisms and risks is required

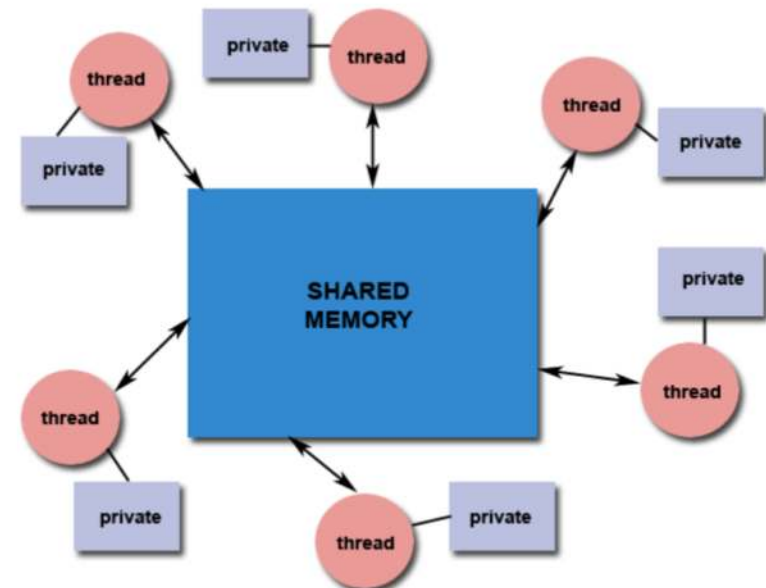


Pthreads Memory Model

The most attractive aspect of threads is the efficiency and easy with which they can share data

```
1  #include "csapp.h"
2  #define N 2
3  void *thread(void *vargp);
4
5  char **ptr; /* Global variable */
6
7  int main()
8  {
9      int i;
10     pthread_t tid;
11     char *msgs[N] = {
12         "Hello from foo",
13         "Hello from bar"
14     };
15
16     ptr = msgs;
17     for (i = 0; i < N; i++)
18         Pthread_create(&tid, NULL, thread, (void *)i);
19     Pthread_exit(NULL);
20 }
21
22 void *thread(void *vargp)
23 {
24     int myid = (int)vargp;
25     static int cnt = 0;
26     printf("[%d]: %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
27     return NULL;
28 }
```

- Thread context: TID, stack, stack pointer (SP), program counter (PC), condition codes (CC), registers values.
- Shared: process virtual address space → code, read/write data, heap, shared libraries, open files.



Mapping variables to memory

```
1  #include "csapp.h"
2  #define N 2
3  void *thread(void *vargp);
4
5  char **ptr; /* Global variable */
6
7  int main()
8  {
9      int i;
10     pthread_t tid;
11     char *msgs[N] = {
12         "Hello from foo",
13         "Hello from bar"
14     };
15
16     ptr = msgs;
17     for (i = 0; i < N; i++)
18         Pthread_create(&tid, NULL, thread, (void *)i);
19     Pthread_exit(NULL);
20 }
21
22 void *thread(void *vargp)
23 {
24     int myid = (int)vargp;
25     static int cnt = 0;
26     printf("[%d]: %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
27     return NULL;
28 }
```

- *global* variables: declared outside functions, only one copy in virtual memory area, readable/writable by any thread.
- *local automatic* variables: declared inside functions, each thread's stack contains its own instance of it. E.g. `tid` and `myid`.
- *local static* variables: declared inside functions with the `static` attribute. There is only one copy in virtual memory area, readable/writable by any thread.
- *shared* variables: a variable is shared only and only if one of its instances is referenced by more than one thread. E.g. `cnt`.



Thread Synchronization

Synchronization Pitfalls

```
3 void *thread(void *vargp); /* Thread routine prototype */
4
5
6 volatile int cnt = 0; /* Counter */
7
8 int main(int argc, char **argv)
9 {
10     int niters;
11     pthread_t tid1, tid2;
12
13     /* Check input argument */
14     if (argc != 2) {
15         printf("usage: %s <niters>\n", argv[0]);
16         exit(0);
17     }
18     niters = atoi(argv[1]);
19
20     /* Create threads and wait for them to finish */
21     Pthread_create(&tid1, NULL, thread, &niters);
22     Pthread_create(&tid2, NULL, thread, &niters);
23     Pthread_join(tid1, NULL);
24     Pthread_join(tid2, NULL);
25
26     /* Check result */
27     if (cnt != (2 * niters))
28         printf("BOOM! cnt=%d\n", cnt);
29     else
30         printf("OK cnt=%d\n", cnt);
31     exit(0);
32 }
33
34 /* Thread routine */
35 void *thread(void *vargp)
36 {
37     int i, niters = *((int *)vargp);
38
39     for (i = 0; i < niters; i++)
40         cnt++;
41
42     return NULL;
43 }
```

What do you expect the code is going to print when executed on a multiprocessor?

Synchronization Pitfalls



```
3 void *thread(void *vargp); /* Thread routine prototype */
4
5
6 volatile int cnt = 0; /* Counter */
7
8 int main(int argc, char **argv)
9 {
10     int niters;
11     pthread_t tid1, tid2;
12
13     /* Check input argument */
14     if (argc != 2) {
15         printf("usage: %s <niters>\n", argv[0]);
16         exit(0);
17     }
18     niters = atoi(argv[1]);
19
20     /* Create threads and wait for them to finish */
21     Pthread_create(&tid1, NULL, thread, &niters);
22     Pthread_create(&tid2, NULL, thread, &niters);
23     Pthread_join(tid1, NULL);
24     Pthread_join(tid2, NULL);
25
26     /* Check result */
27     if (cnt != (2 * niters))
28         printf("BOOM! cnt=%d\n", cnt);
29     else
30         printf("OK cnt=%d\n", cnt);
31     exit(0);
32 }
33
34 /* Thread routine */
35 void *thread(void *vargp)
36 {
37     int i, niters = *((int *)vargp);
38
39     for (i = 0; i < niters; i++)
40         cnt++;
41
42     return NULL;
43 }
```

What do you expect the code is going to print when executed on a uniprocessor?

Synchronization Pitfalls



```
3 void *thread(void *vargp); /* Thread routine prototype */
4
5
6 volatile int cnt = 0; /* Counter */
7
8 int main(int argc, char **argv)
9 {
10     int niters;
11     pthread_t tid1, tid2;
12
13     /* Check input argument */
14     if (argc != 2) {
15         printf("usage: %s <niters>\n", argv[0]);
16         exit(0);
17     }
18     niters = atoi(argv[1]);
19
20     /* Create threads and wait for them to finish */
21     Pthread_create(&tid1, NULL, thread, &niters);
22     Pthread_create(&tid2, NULL, thread, &niters);
23     Pthread_join(tid1, NULL);
24     Pthread_join(tid2, NULL);
25
26     /* Check result */
27     if (cnt != (2 * niters))
28         printf("BOOM! cnt=%d\n", cnt);
29     else
30         printf("OK cnt=%d\n", cnt);
31     exit(0);
32 }
33
34 /* Thread routine */
35 void *thread(void *vargp)
36 {
37     int i, niters = *((int *)vargp);
38
39     for (i = 0; i < niters; i++)
40         cnt++;
41
42     return NULL;
43 }
```

Result

```
linux> ./badcnt 1000000
BOOM! cnt=1445085
```

```
linux> ./badcnt 1000000
BOOM! cnt=1915220
```

```
linux> ./badcnt 1000000
BOOM! cnt=1404746
```

What is going on?

Synchronization Pitfalls



```
3 void *thread(void *vargp); /* Thread routine prototype */
4
5
6 volatile int cnt = 0; /* Counter */
7
8 int main(int argc, char **argv)
9 {
10     int niters;
11     pthread_t tid1, tid2;
12
13     /* Check input argument */
14     if (argc != 2) {
15         printf("usage: %s <niters>\n", argv[0]);
16         exit(0);
17     }
18     niters = atoi(argv[1]);
19
20     /* Create threads and wait for them to finish */
21     Pthread_create(&tid1, NULL, thread, &niters);
22     Pthread_create(&tid2, NULL, thread, &niters);
23     Pthread_join(tid1, NULL);
24     Pthread_join(tid2, NULL);
25
26     /* Check result */
27     if (cnt != (2 * niters))
28         printf("BOOM! cnt=%d\n", cnt);
29     else
30         printf("OK cnt=%d\n", cnt);
31     exit(0);
32 }
33
34 /* Thread routine */
35 void *thread(void *vargp)
36 {
37     int i, niters = *((int *)vargp);
38
39     for (i = 0; i < niters; i++)
40         cnt++;
41
42     return NULL;
43 }
```

Where is the variable `cnt` mapped to in memory?
Is `cnt` private or shared?

Synchronization Pitfalls



```
3 void *thread(void *vargp); /* Thread routine prototype */
4
5
6 volatile int cnt = 0; /* Counter */
7
8 int main(int argc, char **argv)
9 {
10     int niters;
11     pthread_t tid1, tid2;
12
13     /* Check input argument */
14     if (argc != 2) {
15         printf("usage: %s <niters>\n", argv[0]);
16         exit(0);
17     }
18     niters = atoi(argv[1]);
19
20     /* Create threads and wait for them to finish */
21     Pthread_create(&tid1, NULL, thread, &niters);
22     Pthread_create(&tid2, NULL, thread, &niters);
23     Pthread_join(tid1, NULL);
24     Pthread_join(tid2, NULL);
25
26     /* Check result */
27     if (cnt != (2 * niters))
28         printf("BOOM! cnt=%d\n", cnt);
29     else
30         printf("OK cnt=%d\n", cnt);
31     exit(0);
32 }
33
34 /* Thread routine */
35 void *thread(void *vargp)
36 {
37     int i, niters = *((int *)vargp);
38
39     for (i = 0; i < niters; i++)
40         cnt++;
41
42     return NULL;
43 }
```

C code for thread i

```
for (i=0; i < niters; i++)
    cnt++;
```



Asm code for thread i

movl (%rdi),%ecx movl \$0,%edx cmpl %ecx,%edx jge .L13	} H_i : Head	
.L11: movl cnt(%rip),%eax incl %eax movl %eax,cnt(%rip)		} L_i : Load cnt U_i : Update cnt S_i : Store cnt
incl %edx cmpl %ecx,%edx jle .L11 .L13:		

- Here the subscript " i " refers to thread with TID = i .
- The head (H_i) and tail (T_i) codes manipulate only local stack variables (private).
- L_i , U_i , and S_i load, update and store back the shared counter cnt.

A note on the volatile qualifier.

integer n $\leftarrow$ 0	
p	q
integer local1, local2	integer local
p1: n $\leftarrow$ <i>some expression</i>	q1: local $\leftarrow$ n + 6
p2: <i>computation not using n</i>	q2:
p3: local1 $\leftarrow$ (n + 5) * 7	q3:
p4: local2 $\leftarrow$ n + 5	q4:
p5: n $\leftarrow$ local1 * local2	q5:

- A variable should be declared `volatile` whenever its value could change unexpectedly.
- The `q1` statement in thread **q** can be interleaved at any place during the execution of thread **p**.
- Because of compiler optimizations, **q** may not use the *most recent* value of `n` for `q1`. E. g. the compiler may decide to keep `n` in a register throughout `p1` to `p4`.
- Specifying a variable as `volatile` instructs the compiler to load and store the value of the variable at each use, rather than attempt to optimize away these loads and stores.

Synchronization Pitfalls



```
3 void *thread(void *vargp); /* Thread routine prototype */
4
5
6 volatile int cnt = 0; /* Counter */
7
8 int main(int argc, char **argv)
9 {
10     int niters;
11     pthread_t tid1, tid2;
12
13     /* Check input argument */
14     if (argc != 2) {
15         printf("usage: %s <niters>\n", argv[0]);
16         exit(0);
17     }
18     niters = atoi(argv[1]);
19
20     /* Create threads and wait for them to finish */
21     Pthread_create(&tid1, NULL, thread, &niters);
22     Pthread_create(&tid2, NULL, thread, &niters);
23     Pthread_join(tid1, NULL);
24     Pthread_join(tid2, NULL);
25
26     /* Check result */
27     if (cnt != (2 * niters))
28         printf("BOOM! cnt=%d\n", cnt);
29     else
30         printf("OK cnt=%d\n", cnt);
31     exit(0);
32 }
33
34 /* Thread routine */
35 void *thread(void *vargp)
36 {
37     int i, niters = *((int *)vargp);
38
39     for (i = 0; i < niters; i++)
40         cnt++;
41
42     return NULL;
43 }
```

Why this code does not give the correct result on a uniprocessor?

C code for thread i

```
for (i=0; i < niters; i++)
    cnt++;
```



Asm code for thread i

movl (%rdi),%ecx	} H_i : Head	
movl \$0,%edx		
cmpl %ecx,%edx		
jge .L13		
.L11:		} L_i : Load cnt U_i : Update cnt S_i : Store cnt
movl cnt(%rip),%eax		
incl %eax		
movl %eax,cnt(%rip)		
.L12:		} T_i : Tail
incl %edx		
cmpl %ecx,%edx		
jle .L11		
.L13:		

Synchronization Pitfalls



```
3 void *thread(void *vargp); /* Thread routine prototype */
4
5
6 volatile int cnt = 0; /* Counter */
7
8 int main(int argc, char **argv)
9 {
10     int niters;
11     pthread_t tid1, tid2;
12
13     /* Check input argument */
14     if (argc != 2) {
15         printf("usage: %s <niters>\n", argv[0]);
16         exit(0);
17     }
18     niters = atoi(argv[1]);
19
20     /* Create threads and wait for them to finish */
21     Pthread_create(&tid1, NULL, thread, &niters);
22     Pthread_create(&tid2, NULL, thread, &niters);
23     Pthread_join(tid1, NULL);
24     Pthread_join(tid2, NULL);
25
26     /* Check result */
27     if (cnt != (2 * niters))
28         printf("BOOM! cnt=%d\n", cnt);
29     else
30         printf("OK cnt=%d\n", cnt);
31     exit(0);
32 }
33
34 /* Thread routine */
35 void *thread(void *vargp)
36 {
37     int i, niters = *((int *)vargp);
38
39     for (i = 0; i < niters; i++)
40         cnt++;
41
42     return NULL;
43 }
```

C code for thread i

```
for (i=0; i < niters; i++)
    cnt++;
```



Asm code for thread i

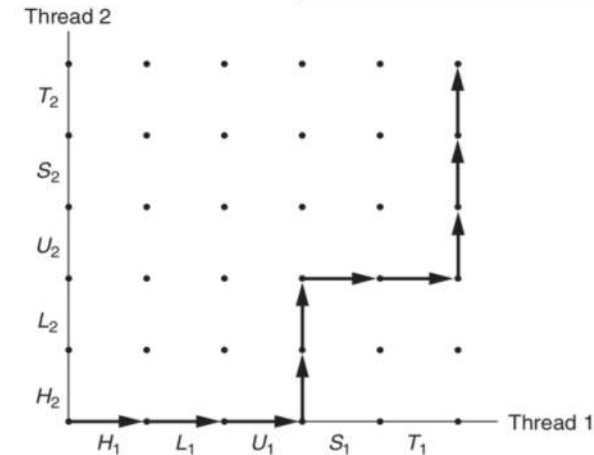
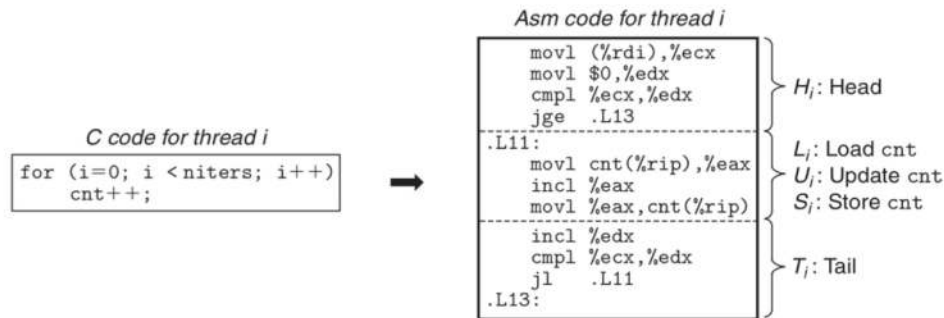
movl (%rdi),%ecx	} H_i : Head	
movl \$0,%edx		
cmpl %ecx,%edx		
jge .L13		
.L11:		} L_i : Load cnt U_i : Update cnt S_i : Store cnt
movl cnt(%rip),%eax		
incl %eax		
movl %eax,cnt(%rip)		
.L13:		} T_i : Tail
incl %edx		
cmpl %ecx,%edx		
jle .L11		

- The OS will run threads concurrently: on a uniprocessor instructions will be interleaved.
- Some of the interleaving ordering will produce the correct results, others will not.
- What about a multiprocessor execution?

State Diagrams



A *state diagram* models the execution of n concurrent execution flows as a trajectory through an n -dimensional Cartesian space (only uniprocessor!)

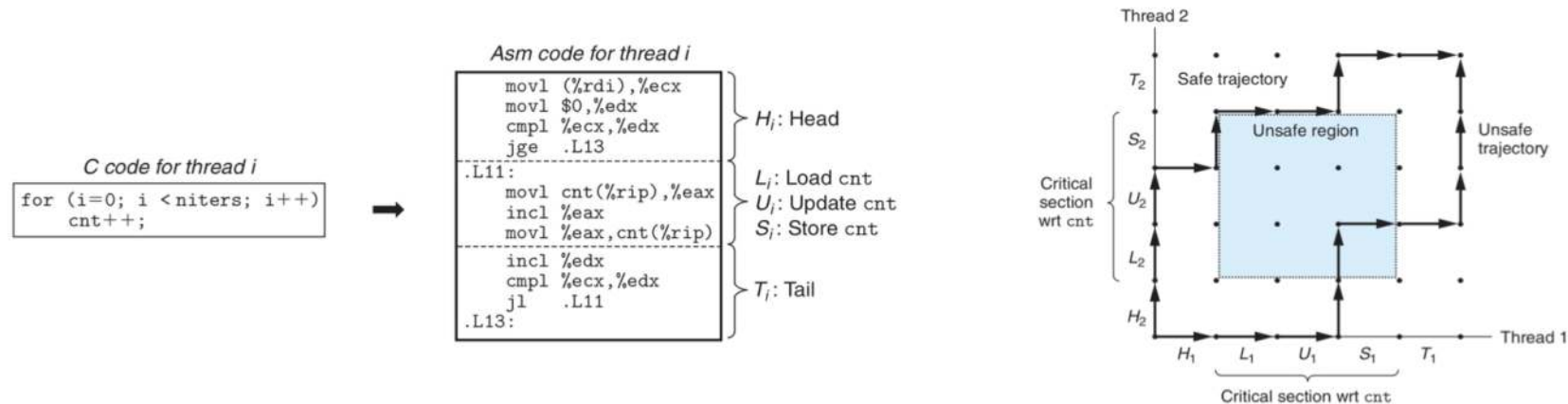


- For thread i the instructions (L_i, U_i, S_i) that manipulate the contents of the shared variable `cnt` constitute a *critical section*
- The instructions of a critical section must be all executed by a single thread at a time.

State Diagrams



A *state diagram* models the execution of n concurrent execution flows as a trajectory through an n -dimensional Cartesian space (only uniprocessor!)



- In order to obtain correct results, each threads must have *mutually exclusive* access to the shared variable while it is executing instructions in the critical section (*mutual exclusion*).
- A trajectory that skirts the *unsafe region* will not cause run time errors (*safe trajectory*).
- In order to guarantee correct execution, we must synchronize threads so that they take safe trajectories.

The Critical Section Problem



Specification of the structure of the critical section problem:

global variables	
p	q
local variables	local variables
loop forever	loop forever
non-critical section	non-critical section
preprotocol	preprotocol
critical section	critical section
postprotocol	postprotocol

- Each of N processes is executing in a infinite loop a sequence of statements that can be divided into two subsequences: the critical section and the noncritical section.
- Correctness specifications:
 - **Mutual exclusion:** Statements from the critical sections of two or more processes must not be interleaved.
 - **Freedom from deadlock:** If some processes are trying to enter their critical sections, then one of them must eventually succeed.
 - **Freedom from starvation:** If any process tries to enter its critical section, then that process must eventually succeed.

The Critical Section Problem

Specification of the structure of the critical section problem:

global variables	
p	q
local variables loop forever non-critical section preprotocol critical section postprotocol	local variables loop forever non-critical section preprotocol critical section postprotocol

- The synchronization mechanism ensures correctness.
- It uses statements places before and after the critical section, called *preprotocol* and *postprotocol*, respectively.
- **Assumption:** assignment statements are *atomic statements*, as are evaluations of boolean conditions in control statements.
- An *atomic statement* is executed to completion without the possibility of interleaving/interrupt from another thread.

Dekker's Algorithm



Youtube video: <https://www.youtube.com/watch?v=MqnplwN7dz0>

Algorithm 3.10: Dekker's algorithm	
boolean wantp $\leftarrow$ false, wantq $\leftarrow$ false integer turn $\leftarrow$ 1	
p	q
loop forever	loop forever
p1: non-critical section	q1: non-critical section
p2: wantp $\leftarrow$ true	q2: wantq $\leftarrow$ true
p3: while wantq	q3: while wantp
p4: if turn = 2	q4: if turn = 1
p5: wantp $\leftarrow$ false	q5: wantq $\leftarrow$ false
p6: await turn = 1	q6: await turn = 2
p7: wantp $\leftarrow$ true	q7: wantq $\leftarrow$ true
p8: critical section	q8: critical section
p9: turn $\leftarrow$ 2	q9: turn $\leftarrow$ 1
p10: wantp $\leftarrow$ false	q10: wantq $\leftarrow$ false

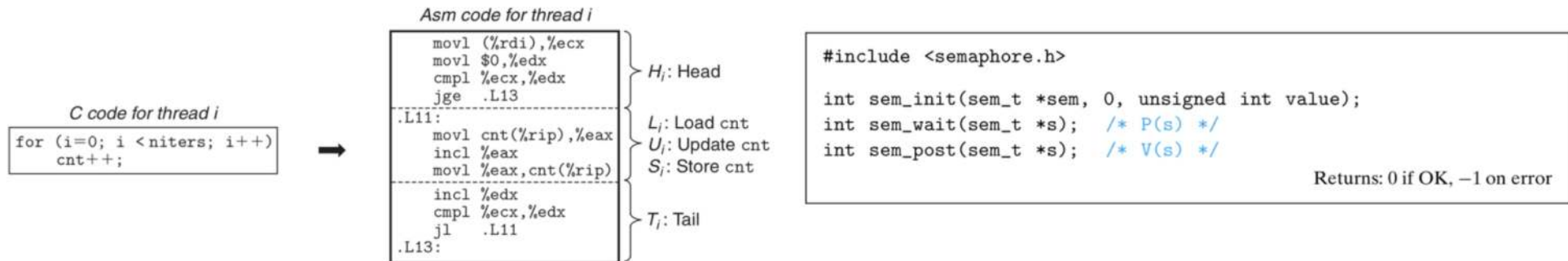
- The variables wantp and wantq ensure mutual exclusion.
- Suppose **p** detects contention by finding wantp == true (p3): it will consult the shared variable turn, to check whether it is its turn (turn == 1) to insist on entering its critical section.
- If so, it executes the loop at p3 and p4, called a *busy-wait loop*, until **q** resets wantq to false, either by terminating its critical section at q10 or by deferring in q5.
- If not, **p** will reset wantp to false and defer to thread **q**, waiting until **q** changes the value of turn after executing its critical section.

Semaphores, Mutexes and Locks



A *semaphore* s is a global variable with a nonnegative integer value that can only be manipulated by two special operations, P and V :

- $P(s)$: if $s \neq 0$, then P decrements s and returns. If $s == 0$, then *suspend* the thread until $s \neq 0$, and the thread is restarted by a V operation.
- $V(s)$: increments s by 1. If there are threads blocked at P , then V restarts exactly one of these thread, which can complete its P operation. The specific thread restarted by V is non-deterministic.
- The test and decrement operations in P and occur and the increment in V occurs atomically (V loads, increments, and stores the semaphore without interruption).



Semaphores, Mutexes and Locks

```
#include <semaphore.h>

int sem_init(sem_t *sem, 0, unsigned int value);
int sem_wait(sem_t *s); /* P(s) */
int sem_post(sem_t *s); /* V(s) */
```

Returns: 0 if OK, -1 on error

```
#include "csapp.h"

void P(sem_t *s); /* Wrapper function for sem_wait */
void V(sem_t *s); /* Wrapper function for sem_post */
```

Returns: nothing

- Programs perform *P* and *V* operations by calling the `sem_wait` and `sem_post` functions, respectively.
- We can associate a semaphore *s*, initially with value 1, with one or multiple shared variables.

```
volatile int cnt = 0; /* Counter */
sem_t mutex;          /* Semaphore that protects counter */

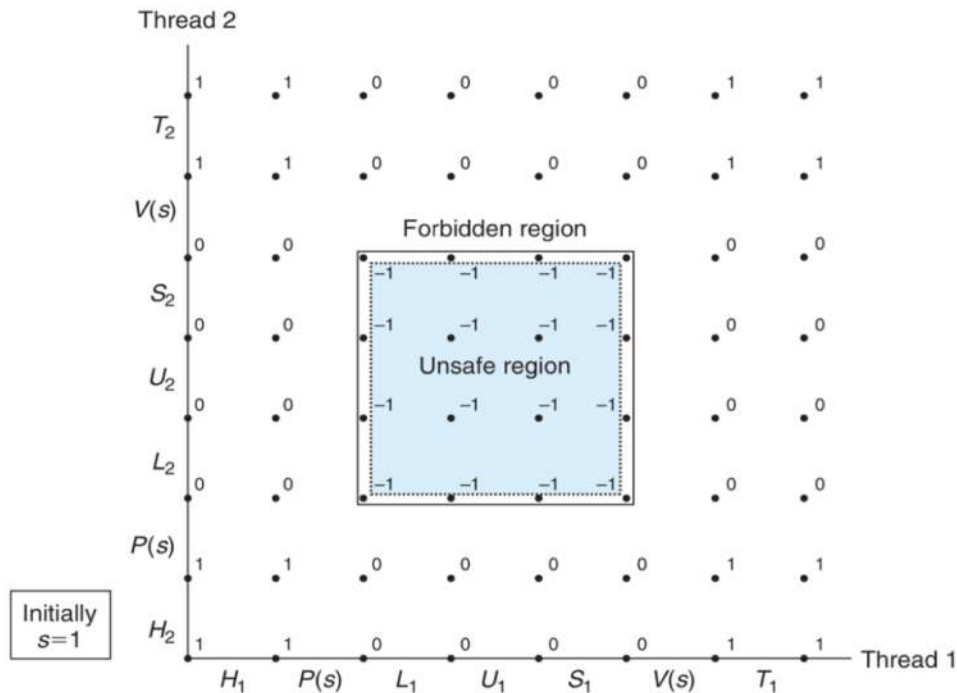
Sem_init(&mutex, 0, 1); /* mutex = 1 */
for (i = 0; i < niters; i++) {
    P(&mutex);
    cnt++;
    V(&mutex);
}
```

- We surround the critical section with *P* and *V* operations.
- Such a binary semaphore whose purpose is to provide mutual exclusion is called a *mutex*.
- Performing a *P* operation on a mutex is called *locking* the mutex.
- Similarly, performing the *V* operation is called *unlocking* the mutex.

Semaphores, Mutexes and Locks

```
volatile int cnt = 0; /* Counter */
sem_t mutex;          /* Semaphore that protects counter */

Sem_init(&mutex, 0, 1); /* mutex = 1 */
for (i = 0; i < niters; i++) {
    P(&mutex);
    cnt++;
    V(&mutex);
}
```



- Performing a P operation on a mutex is called *locking* the mutex.
- Similarly, performing the V operation is called *unlocking* the mutex.
- A thread that has locked but not yet unlocked a mutex is said to be *holding* the mutex.
- This combination of initial value of s , P and V operations creates a collection of states, called a forbidden region, where $s < 0$.
- The forbidden region completely encloses the unsafe region, no feasible trajectory can touch any part of the unsafe region, leading to correct execution.

Mutexes and Locks with Pthreads

Pthreads provides primitives directly for creating, locking and unlocking mutexes.

```
#include <pthread.h>

int pthread_mutex_init(pthread_mutex_t *mutex,
    const pthread_mutexattr_t *attr);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

```
4  /*
5   * Define a structure, with a mutex.
6   */
7  typedef struct my_struct_tag {
8      pthread_mutex_t  mutex; /* Protects access to value */
9      int              value; /* Access protected by mutex */
10 } my_struct_t;
11
12 int main (int argc, char *argv[])
13 {
14     my_struct_t *data;
15     int status;
16
17     data = malloc (sizeof (my_struct_t));
18     if (data == NULL)
19         errno_abort ("Allocate structure");
20     status = pthread_mutex_init (&data->mutex, NULL);
21     if (status != 0)
22         err_abort (status, "Init mutex");
23     status = pthread_mutex_destroy (&data->mutex);
24     if (status != 0)
25         err_abort (status, "Destroy mutex");
26     (void)free (data);
27     return status;
28 }
```

- A mutex is represented in your program by a variable of type `pthread_mutex_t`, which must be initialized before usage.
- You should never copy a mutex (result is undefined), if necessary you can copy a pointer to a mutex.
- When you declare a static mutex that has default attributes, you should use the `PTHREAD_MUTEX_INITIALIZER` macro.
- Often you cannot initialize a mutex statically, for example, when you use `malloc` to create a structure that contains a mutex. Then you can use dynamic initialization (`pthread_mutex_init`).
- When you no longer need a mutex that you dynamically initialized, you should destroy the mutex by calling `pthread_mutex_destroy`.

Mutexes and Locks with Pthreads



```
#include <pthread.h>

int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);

#include <stdio.h>
#include <pthread.h>

#define NTHREADS 10
void *thread_function(void *);
pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

main()
{
    pthread_t thread_id[NTHREADS];
    int i, j;

    for(i=0; i < NTHREADS; i++)
    {
        pthread_create( &thread_id[i], NULL, thread_function, NULL );
    }

    for(j=0; j < NTHREADS; j++)
    {
        pthread_join( thread_id[j], NULL);
    }

    /* Now that all threads are complete I can print the final result. */
    /* Without the join I could be printing a value before all the threads */
    /* have been completed. */

    printf("Final counter value: %d\n", counter);
}

void *thread_function(void *dummyPtr)
{
    printf("Thread number %ld\n", pthread_self());
    pthread_mutex_lock( &mutex1 );
    counter++;
    pthread_mutex_unlock( &mutex1 );
}
```

- The `pthread_mutex_lock` routine is used by a thread to acquire a lock on the specified mutex variable. If the mutex is already locked by another thread, this call will block the calling thread until the mutex is unlocked.
- Occasionally you want your code to take some alternate path if the mutex is locked. Your program may be able to do useful work instead of waiting. The `pthread_mutex_trylock` function, which will return an error status (EBUSY) instead of blocking if the mutex is already locked.
- When a thread holding the mutex has executed the code within a critical section, it must unlock the mutex by calling `pthread_mutex_unlock`.

- Thread execution model
 - main thread, peer thread, thread content switch
- Pthreads API
 - Pthread_create, pthread_exit, pthread_join, pthread_cancel, pthread_detach
- Thread memory model
 - Shared: code, heap, shared libraries
 - Private: thread context (TID, stack, program counter ...)
- Thread synchronization
 - Critical section
 - Safe/unsafe trajectory
 - Dekker's algorithm
 - Semaphore
 - Mutexes and locks